

1. JWT Structure Overview

Basic Format

A JSON Web Token consists of three distinct parts separated by dots (.):

HEADER . PAYLOAD . SIGNATURE

Example Token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  
.  
eyJ1c2VySWQiOiJEsInJvbGUiOiJlc2VyIn0  
.  
abc123xyz456signature
```

Note: Each dot separates a different component of the token.

2. Component Breakdown

2.1 Header

The header contains metadata about the token.

Original JSON Representation:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Field	Description	Possible Values
alg	Signing algorithm used	HS256, RS256, ES256
typ	Token type	JWT

Encoded Form (Base64Url):

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

This represents the first part of the token.

2.2 Payload

The payload contains the actual user data and claims.

Original JSON Representation:

```
{
  "userId": 1,
  "role": "user"
}
```

Additional standard claims may include:

Claim	Name	Description
iat	Issued At	Timestamp of token creation
exp	Expiration Time	Token expiry timestamp
iss	Issuer	Identifies who issued the token
sub	Subject	Identifies the user

Encoded Form (Base64Url):

eyJ1c2VySWQiOjEsInJvbGUiOiJ1c2VyIn0

This represents the second part of the token.

2.3 Signature

The signature is a cryptographic hash used to verify token authenticity.

Example Signature:

abc123xyz456signature

This represents the third part of the token.

3. Signature Creation Process

3.1 Required Components

To create a signature, the server needs:

Component	Source	Example
Header	Encoded from JSON	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
Payload	Encoded from JSON	eyJ1c2VySWQiOjEsInJvbGUiOiJ1c2VyIn0
Secret Key	Server environment variable	mySuperSecretKey123

3.2 Signature Generation Steps

Step 1: Combine Header and Payload

The JWT library creates a string by joining the encoded header and payload with a dot:

```
data = base64UrlEncode(header) + "." + base64UrlEncode(payload)
```

Example Result:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEsInJvbGUiOiJ1c2VyIn0
```

Step 2: Apply Cryptographic Hashing

The library applies the HMACSHA256 algorithm:

```
signature = HMACSHA256(  
    data,                                     // Header.Payload  
    JWT_SECRET                               // mySuperSecretKey123  
) ;
```

Step 3: Encode the Result

The resulting hash is encoded in Base64Url format to produce the final signature.

3.3 Complete Token Assembly

The server combines all three parts:

```
header + "." + payload + "." + signature
```

Final Token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  
.eyJ1c2VySWQiOiJEsInJvbGUiOiJ1c2VyIn0  
.abc123xyz456signature
```

This complete string is sent to the client.

4. Token Verification Mechanism

4.1 What Happens When a Token is Received

When the server receives a request with an Authorization header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEsInJvbmGUiOiJlc2VyIn0.abc123xyz456signature

The verification process begins.

4.2 Verification Steps

Step 1: Split the Token

```
const parts = token.split('.');
```

Result:

Part	Index	Contains
Header	parts[0]	Encoded header
Payload	parts[1]	Encoded payload
Signature	parts[2]	Provided signature

Step 2: Recreate the Signature

The server recreates the signature using:

```
const data = parts[0] + '.' + parts[1];  
const expectedSignature = HMACSHA256(data, JWT_SECRET);
```

Step 3: Compare Signatures

```
if (expectedSignature === parts[2]) {  
    // Token is valid and untampered  
} else {  
    // Token has been modified or is invalid
```

```
}
```

4.3 What Happens When Someone Tamper with the Token

Scenario: Attacker modifies the payload

Original Payload:

```
{  
  "userId": 1,  
  "role": "user"  
}
```

Modified Payload by Attacker:

```
{  
  "userId": 1,  
  "role": "admin"  
}
```

Result:

Component	Original	After Modification
Header	Same	Same
Payload	eyJ1c2VySWQiOiJ1c2VyIn0	NEW_ENCODED_PAYLOAD
Signature	abc123xyz456signature	STILL OLD SIGNATURE

Verification Process:

1. Server receives tampered token
2. Server extracts header and NEW payload
3. Server generates signature from header + new payload + secret key
4. Generated signature will NOT match the old signature
5. Server rejects the token as invalid

Result: Authentication fails, access denied.

5. Security Considerations

5.1 JWT is Not Encrypted by Default

Important: By default, JWT is encoded, not encrypted.

Anyone who obtains a JWT can decode and read its contents using online tools.

Example of readable payload:

```
json
{
  "userId": 1,
  "email": "padma@gmail.com",
  "role": "user",
  "company_id": 1
}
```

Implications:

- Do NOT store sensitive information in JWT payload
- Passwords, credit card numbers, and personal identifiable information should never be placed in a JWT
- Treat JWT payload as publicly readable data

5.2 What the Signature Protects

Can Read	Cannot Do
Header contents	Modify header without detection
Payload contents	Modify payload without detection
Algorithm type	Create valid signature without secret
Claims data	Forge a new valid token

5.3 Security Best Practices

Practice	Recommendation
Secret Key	Store in environment variables, never in code
Key Strength	Use minimum 32-character random strings
Token Expiry	Set reasonable expiration times
Payload Content	Minimize sensitive data
Transport	Always use HTTPS
Storage	Use HttpOnly cookies for web applications

6. Common Misconceptions

Misconception 1: The Secret is Inside the Token

Incorrect Belief:

text
Token contains the secret key used for verification

Reality:

text
Secret key is NEVER inside the token.
It exists only on the server.

Misconception 2: JWT is Encrypted

Incorrect Belief:

text
Nobody can read the contents of a JWT

Reality:

text
JWT is base64Url encoded, not encrypted.
Anyone can decode and read the payload.

Misconception 3: Server Extracts Secret from Token

Incorrect Belief:

text

Server reads secret from token to verify signature

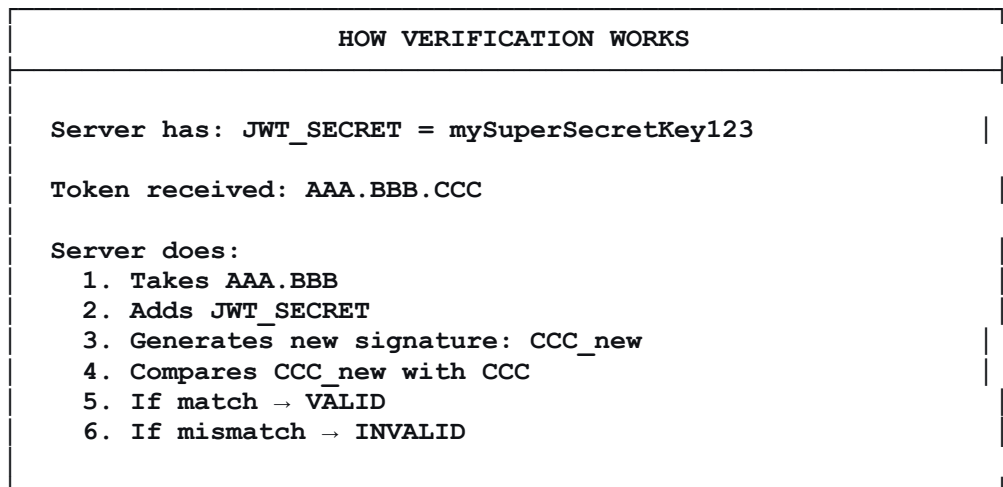
Reality:

text

Server uses its own secret key (stored in environment) to RECREATE the signature and compare.

Correct Understanding:

text



7. Practical Examples

7.1 Real-World Analogy: Stamped Certificate

JWT Component

Analogy

Header + Payload

Certificate content (Name, Role)

Signature

Official school seal/stamp

JWT Component

Analogy

Secret Key

Stamp design (only school knows)

Verification

Checking if seal is authentic

Process:

1. School creates certificate with student details
2. School applies unique seal (signature)
3. Anyone can read certificate details
4. Anyone can see the seal exists
5. Only school can verify if seal is genuine
6. Anyone trying to modify certificate would need the original seal

7.2 Code Example: Token Generation

```
javascript
const jwt = require('jsonwebtoken');

// Server secret - never exposed to client
const JWT_SECRET = process.env.JWT_SECRET; // "mySuperSecretKey123"

// User data to encode
const payload = {
  userId: 1,
  role: 'user',
  email: 'padma@gmail.com'
};

// Generate token
const token = jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' });

console.log(token);
// Output: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJlbnV4cCI6MTc4MTA3NTA5N30.signature_here
```

7.3 Code Example: Token Verification

```
javascript
const jwt = require('jsonwebtoken');

// Token received from client
const token = req.headers.authorization.split(' ')[1];

try {
  // Verify using the SAME secret key
```

```

    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    console.log('Token is valid!');
    console.log('Decoded payload:', decoded);
    // Output: { userId: 1, role: 'user', email: 'padma@gmail.com', iat: 1780
470297, exp: 1781075097 }

    req.user = decoded;
    next();
  } catch (error) {
    console.log('Token verification failed!');
    res.status(401).json({ error: 'Invalid token' });
  }
}

```

7.4 What an Attacker Sees

Attacker obtains token:

```

text
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJlbnVjbGUiOiJ1c2VyIn0.abc1
23signature

```

Attacker decodes payload (using online tool):

```

json
{
  "userId": 1,
  "role": "user"
}

```

Attacker attempts to change role to admin:

```

json
{
  "userId": 1,
  "role": "admin"
}

```

Attacker re-encodes and creates new token:

```

text
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJlbnVjbGUiOiJhZG1pbiJ9.OLD
_SIGNATURE

```

Server verification:

Step	Result
Extract header + payload	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pbiJ9
Generate new signature	NEW_SIGNATURE (different from OLD_SIGNATURE)
Compare signatures	MISMATCH
Verdict	REJECTED - Invalid Token

8. Summary

Key Points to Remember

Concept	Explanation
JWT Structure	Header.Payload.Signature
Signature Purpose	Proves payload hasn't been altered
Secret Key Location	Server only, never in token
Verification Method	Server recreates signature and compares
Payload Visibility	Anyone can read the payload
Tamper Protection	Any modification breaks the signature

Verification Flow

```

text
Step 1: Receive token from client
Step 2: Split token into header, payload, signature

```

Step 3: Combine header and payload
Step 4: Generate new signature using server secret
Step 5: Compare new signature with received signature
Step 6: If match → Valid token
Step 7: If mismatch → Invalid token